

Labo 00 — Les fondamentaux Linux : ce que Docker suppose que vous savez

Théorie — le noyau, les processus, les utilisateurs, les fichiers, le shell et trois notions de réseau. Tout ce que les labos suivants utiliseront sans plus jamais l'expliquer.

Objectifs

- Situer les trois étages d'un système Linux : noyau, appels système, espace utilisateur.
- Décrire un processus : PID, parent, environnement, signaux, code de sortie.
- Lire une ligne de `ls -l` : propriétaire, groupe, permissions — et savoir ce que `root` peut faire de plus que vous.
- Utiliser le shell comme un outil : variables d'environnement, `PATH`, redirections, pipes.
- Relier chaque notion à ce qu'elle deviendra dans les labos Docker.

1. Pourquoi un labo Linux dans une formation Docker

Parce qu'un conteneur n'est **rien d'autre que du Linux**. Quand vous lirez, au labo 01, qu'« un conteneur est un processus isolé », la phrase ne vous servira que si *processus* est pour vous une idée précise : quelque chose qui a un numéro, un parent, un environnement, une façon de mourir. De même, le « rootless » de Podman est incompréhensible sans les UID, un port « déjà pris » sans la notion de port, un volume sans la notion de montage. Ce labo installe ce vocabulaire, uniquement lui, et chaque section annonce le labo où la notion resservira.

→ HISTOIRE

Unix naît en 1969 aux Bell Labs ; il impose deux idées qui gouvernent encore tout : *tout est fichier* et *de petits programmes qu'on assemble*. En 1991, un étudiant finlandais, Linus Torvalds, écrit un noyau libre compatible Unix : Linux. Ubuntu (2004) en est une **distribution** : le noyau Linux plus un espace utilisateur choisi et empaqueté. Et depuis 2019, Microsoft livre son propre noyau Linux dans Windows : c'est WSL 2, votre machine de labo.

2. Le noyau et l'espace utilisateur

Un système Linux a deux étages. En bas, le **noyau** (*kernel*) : le seul programme qui touche le matériel. Il crée les processus, distribue le temps CPU et la mémoire, lit les disques, envoie les paquets réseau. En haut, l'**espace utilisateur** (*userland*) : tout le reste — `bash`, `ls`, `java`, votre application. Entre les deux, une frontière unique : les **appels système** (*syscalls*). Un programme ne lit jamais un fichier lui-même ; il demande `open` puis `read` au noyau, qui décide si oui ou non.

Cette frontière explique deux choses que vous verrez sans cesse. D'abord, la portabilité : un binaire Linux fonctionne sur n'importe quelle distribution, car il ne demande que des appels système, identiques partout. Ensuite, le contrôle : puisque *tout* passe par le noyau, il suffit au noyau de mentir un peu (« tu es le processus 1 », « voici ton `/` ») pour isoler un processus — c'est exactement ce que fera un conteneur au labo 01.

→ WINDOWS / WSL

Un programme Linux ne « parle » qu'à un noyau Linux ; Windows a le sien, incompatible. **WSL 2** (*Windows Subsystem for Linux*) résout le problème en faisant tourner un vrai noyau Linux dans une VM minuscule gérée par Windows. Votre Ubuntu 24.04 est une distribution *dans* cette VM : `uname -r` y répond `...microsoft-standard-WSL2`, la signature du noyau compilé par Microsoft. Le disque Windows y est visible sous `/mnt/c`.

3. Les processus

Un **processus** est un programme en cours d'exécution : du code, de la mémoire, et une identité. Le noyau lui donne un **PID** (*process ID*) unique et retient son parent, le **PPID**. Tout processus naît d'un autre — quand vous tapez `ls`, votre shell se duplique (`fork`) puis le clone se remplace par `ls` (`exec`). Au démarrage, le noyau lance un premier processus, **PID 1** (`systemd` sur Ubuntu), ancêtre de tous les autres ; quand un parent meurt avant son enfant, l'orphelin est adopté par PID 1. Retenez ce numéro : dans un conteneur, c'est *votre application* qui sera PID 1, avec des responsabilités inattendues (labo 03).

→ LINUX

Un **daemon** (démon) est un processus de service : lancé au démarrage par `systemd`, détaché de tout terminal, il tourne en arrière-plan en attendant qu'on ait besoin de lui — `sshd` attend les connexions SSH, `cron` l'heure de ses tâches. Par convention, son nom finit par un « d ». Retenez le mot : Docker repose entièrement sur un daemon, `dockerd`, et Podman se définit par son absence — c'est LE débat du labo 01.

Un processus se termine toujours en rendant un **code de sortie** : `0` signifie « succès », tout le reste est un échec. Le shell le stocke dans `$?`. Quelques valeurs conventionnelles : `1` erreur générale, `2` mauvais usage, `126` fichier non exécutable, `127` commande introuvable, `128 + n` mort par le signal *n*.

Car on ne « ferme » pas un processus : on lui envoie un **signal**, une notification numérotée du noyau. Les trois à connaître :

Signal	Numéro	Sens	Le processus peut-il l'ignorer ?
<code>SIGTERM</code>	15	« Termine-toi proprement »	Oui — il peut d'abord sauver, fermer, ranger
<code>SIGKILL</code>	9	Mort immédiate, par le noyau	Non — et rien n'est rangé
<code>SIGINT</code>	2	Interruption clavier (<code>Ctrl+C</code>)	Oui

À RETENIR

`kill` ne veut pas dire « tuer » mais « envoyer un signal » ; par défaut il envoie le poli `SIGTERM`. Un processus tué par `SIGKILL` sort avec le code `137` (`128 + 9`). Vous reverrez ce nombre toute votre vie Docker : c'est la signature d'un conteneur arrêté de force — souvent par manque de mémoire.

Le noyau expose l'état de chaque processus dans `/proc`, un faux répertoire : `/proc/1234/` décrit le processus 1234 (sa commande, son environnement, ses limites), fabriqué à la volée, sans occuper un octet de disque. `ps` ne fait que le lire.

4. Utilisateurs, groupes, permissions

Chaque processus s'exécute *en tant que* quelqu'un : un **utilisateur**, identifié par un numéro, l'**UID**, et des **groupes** (GID). Le noyau ne connaît que les numéros ; les noms (`kevin` , `postgres`) viennent du fichier `/etc/passwd` . Votre premier utilisateur Ubuntu a l'UID **1000**. L'utilisateur `root` , UID **0**, est spécial : le noyau ne lui refuse rien. La commande `sudo` exécute une commande *en tant que* root, en journalisant qui l'a demandé.

Chaque fichier a un propriétaire, un groupe, et neuf bits de permission, lisibles dans `ls -l` :

```
-rw-r----- 1 root shadow 1234 ... /etc/shadow
└┐└┐└┐      └┐ propriétaire root, groupe shadow
  │ │ └┐      └┐ autres : rien
  │ └┐        └┐ groupe shadow : lecture
  └┐          └┐ root : lecture + écriture
```

`r` lire, `w` écrire, `x` exécuter (pour un répertoire : y entrer). `chmod` change ces bits, `chown` le propriétaire. Un détail qui piège tout le monde : un script doit être **exécutable** (`chmod +x`) pour être lancé par `./script.sh` — sinon le shell répond `Permission denied` , code 126.

→ SÉCURITÉ

La règle d'or : on ne travaille jamais en root, on élève ses droits ponctuellement avec `sudo` . C'est la version Linux du principe du moindre privilège, et c'est l'argument central de Podman **rootless** : vos conteneurs tourneront sous l'UID 1000, pas sous l'UID 0, et une application compromise n'aura que vos droits (labo 01).

PIÈGE

Le noyau compare des **numéros**, pas des noms. Un fichier créé par l'UID 1000 dans un conteneur appartient à l'UID 1000 partout, même si le nom affiché change d'un système à l'autre. Cette évidence deviendra le casse-tête classique des volumes au labo 06.

5. Fichiers, arborescence, montages

Sous Unix, *tout est fichier* : les documents, mais aussi les disques (`/dev/sdc`), l'état du noyau (`/proc`), les sockets. Il n'y a pas de lecteurs `C:` ou `D:` : un seul arbre, partant de la racine `/` , où chaque disque ou système de fichiers est **monté** — accroché à un répertoire. `findmnt /` vous dit quel disque fournit la racine ; sur WSL, `/mnt/c` est le montage du disque Windows. Monter, démonter, superposer des systèmes de fichiers : c'est la mécanique exacte des images et des volumes Docker (labos 02 et 06).

Les répertoires standard à reconnaître : `/etc` (configuration), `/home` (vos fichiers), `/usr/bin` (les programmes), `/var` (données qui vivent : logs, bases), `/tmp` (temporaire), `/proc` et `/sys` (fenêtres sur le noyau).

6. Le shell : l'environnement, le PATH, la plomberie

Le **shell** (`bash`) est un processus comme un autre, dont le travail est de lancer les autres. Trois de ses mécanismes sont du pur « savoir Docker ».

Les variables d'environnement. Chaque processus naît avec un dictionnaire clé=valeur hérité de son parent : `HOME` , `PATH` , `LANG` ... Une variable de shell (`MSG=coucou`) reste locale ; elle n'entre dans l'environnement des enfants qu'après `export MSG` . C'est LE canal de configuration des

conteneurs : au labo 08, votre application Spring Boot lira son mot de passe de base de données dans une variable, jamais dans un fichier de l'image.

→ JAVA

Une JVM est un processus ordinaire : `java -jar app.jar` a un PID, un UID, des variables. Spring Boot lit l'environnement au démarrage : `SERVER_PORT=9090` suffit à changer son port, sans toucher au JAR. `System.getenv("HOME")` en Java, c'est la lecture de ce même dictionnaire hérité.

Le PATH. Quand vous tapez `ls`, le shell cherche un exécutable nommé `ls` dans la liste de répertoires de la variable `PATH`, dans l'ordre. `which ls` montre ce qu'il a trouvé ; `command not found` (code 127) signifie « dans aucun de ces répertoires ». C'est pourquoi un script du répertoire courant se lance `./script.sh` : « ici » n'est pas dans le `PATH`, par prudence.

Redirections et pipes. Un processus a trois flux : l'entrée (0, *stdin*), la sortie (1, *stdout*) et l'erreur (2, *stderr*). Le shell les branche où on veut : `> fichier` détourne la sortie, `2>` l'erreur, `2>&1` fusionne les deux, et `commande1 | commande2` branche la sortie de l'une sur l'entrée de l'autre. Vous assemblerez ces tuyaux dans tous les labos (`podman ps | grep ...`), et les logs d'un conteneur ne sont rien d'autre que ses flux 1 et 2 capturés (labo 03).

7. Le réseau en trois notions

Il vous faut trois idées pour survivre jusqu'au labo 07. **L'interface** : la prise réseau d'une machine, avec une adresse IP ; `lo`, l'interface *loopback*, porte l'adresse `127.0.0.1`, alias `localhost` — la machine se parlant à elle-même. **Le port** : un numéro de 1 à 65535 qui distingue les services d'une même adresse ; un seul processus écoute sur un port donné, `ss -tlnp` liste qui écoute où. **Le privilège** : les ports inférieurs à 1024 sont réservés à root — raison pour laquelle votre serveur de test écoutera sur 8080 et non sur 80, et pourquoi Podman rootless refusera `-p 80:80` (labo 07).

→ RÉSEAU

`curl` est le couteau suisse : il fait une requête HTTP et affiche la réponse brute. `curl -i http://localhost:8080/` montre le code (`200 OK`, `404` ...), les en-têtes, le corps. C'est l'outil n°1 pour tester une API conteneurisée sans navigateur.

→ WINDOWS / WSL

WSL 2 relaie automatiquement `localhost` : un serveur qui écoute sur le port 8080 *dans* Ubuntu est joignable depuis un navigateur **Windows** à `http://localhost:8080`. Pratique, mais souvenez-vous que ce relais est une faveur de WSL, pas une propriété de Linux.

8. En entreprise

Tout l'écosystème conteneurs est l'industrialisation de ces notions. Un serveur de production Spring Boot, c'est : un processus `java` (PID) lancé par un utilisateur applicatif sans droits (UID), configuré par variables d'environnement, écrivant ses logs sur *stdout*, écoutant sur le port 8080, arrêté par `SIGTERM` lors des déploiements. L'exploitant qui diagnostique un incident enchaîne `ps`, `ss`, `curl`, lit `$?`, et fouille les logs avec `grep`. Docker ne remplacera rien de tout cela : il l'emballage.

PODMAN

Podman poussera cette logique jusqu'au bout : pas de daemon, juste *votre* utilisateur (UID 1000) qui lance des processus. Tout ce labo — UID, signaux, `/proc`, ports non privilégiés — est la description exacte de ce que Podman a le droit de faire sans `sudo`. Docker, lui, s'appuie sur un daemon tournant en root (`dockerd`) ; la différence occupera le labo 01.

À retenir

- Le **noyau** contrôle tout ; les programmes ne font que des **appels système**. Isoler un processus, c'est faire mentir le noyau — l'idée fondatrice du conteneur.
- Un **processus** a un PID, un parent, un environnement hérité, et finit par un **code de sortie** : `0` = succès, `137` = tué par SIGKILL.
- `SIGTERM` demande poliment, `SIGKILL` exécute sans appel. Un service bien élevé s'arrête sur SIGTERM.
- Le noyau raisonne en **UID/GID** numériques ; `root` = UID 0 = tous les droits ; `sudo` élève ponctuellement.
- Un seul arbre de fichiers ; disques et systèmes virtuels y sont **montés** ; `/proc` est la fenêtre sur le noyau.
- Les **variables d'environnement** passent du parent à l'enfant ; le `PATH` décide quelles commandes existent.
- Un service = une adresse + un **port** ; `localhost` = la machine elle-même ; ports < 1024 réservés à root.

Vocabulaire

noyau / kernel : le programme qui contrôle matériel et processus. — **userland** : tout ce qui tourne au-dessus du noyau. — **appel système** : requête d'un programme au noyau (`open`, `fork` ...). — **processus** : programme en exécution, identifié par un **PID**. — **PID 1** : premier processus, ancêtre et tuteur de tous. — **daemon** : processus de service en arrière-plan, sans terminal, géré par `systemd` (`sshd`, `dockerd`). — **signal** : notification envoyée à un processus (`SIGTERM`, `SIGKILL`). — **code de sortie** : entier rendu à la mort d'un processus, `0` = succès, dans `$?`. — **UID / GID** : numéros d'utilisateur et de groupe, seuls compris du noyau. — **root** : UID 0, aucun contrôle ne s'applique. — **montage** : rattachement d'un système de fichiers à un répertoire de l'arbre. — **/proc** : arborescence virtuelle exposant l'état du noyau et des processus. — **variable d'environnement** : paire clé=valeur héritée par les processus enfants. — **PATH** : liste des répertoires où le shell cherche les commandes. — **stdin / stdout / stderr** : les trois flux standard (0, 1, 2). — **pipe** : branchement de la sortie d'un processus sur l'entrée d'un autre. — **port** : numéro identifiant un service sur une adresse IP. — **localhost** : `127.0.0.1`, l'adresse de la machine elle-même.